CS218
Programmable & Interoperable Blockchain

Project 6
Decentralised Identity Verification

Kredent: Blockchain Based KYC & Identity Verification System

Submitted By

Name	Roll Number
Devanshu Dubey	240001024
Abhinav Patel	240001004
Abhay Lodhi	240001003
Pratyush Gupta	240001054
Abhishek Kumar Verma	240001005
Vivek Sahu	240005051

Submitted To

Prof. Subhra Mazumdar

Abstract

This project presents a decentralised identity verification platform named **Kredent**, designed using Ethereum smart contracts, React frontend, Express backend, and MongoDB storage. The system follows a hybrid architecture where sensitive identity documents remain encrypted off-chain while only cryptographic hashes are stored on-chain.

The platform uses role-based access control through OpenZeppelin AccessControl contracts to manage administrators and verifiers securely. The project demonstrates GDPR-compliant identity verification using hash-only blockchain storage and composability through a KYC-gated auction smart contract.

The project also integrates MetaMask wallet connectivity, AES-256 encryption, gas optimisation strategies, automated testing, and smart contract security mechanisms such as reentrancy protection.

Contents

1. Introduction

Blockchain technology enables decentralised trustless systems where transactions and records become immutable and transparent. However, storing sensitive personal information directly on-chain violates privacy regulations such as GDPR.

This project solves the issue by implementing a decentralised identity verification system that stores only cryptographic hashes on-chain while keeping encrypted documents off-chain.

1.1 Problem Statement

Traditional KYC systems suffer from:

- Centralised storage vulnerabilities
- Data leakage risks
- Poor transparency
- Lack of user ownership
- GDPR compliance issues

1.2 Objectives

- Build a decentralised identity verification platform
- Store only keccak256 hashes on-chain
- Maintain GDPR compliance
- Implement verifier-based approval system
- Demonstrate composability using KYC gated auction
- Integrate MetaMask and React frontend

2. System Architecture

The project consists of three major components:

1. Frontend Application
2. Backend Server
3. Blockchain Smart Contracts

2.1 Frontend

The frontend is built using React and Vite. It provides:

- MetaMask wallet integration
- User dashboard
- Admin dashboard
- Verifier dashboard
- Auction interface

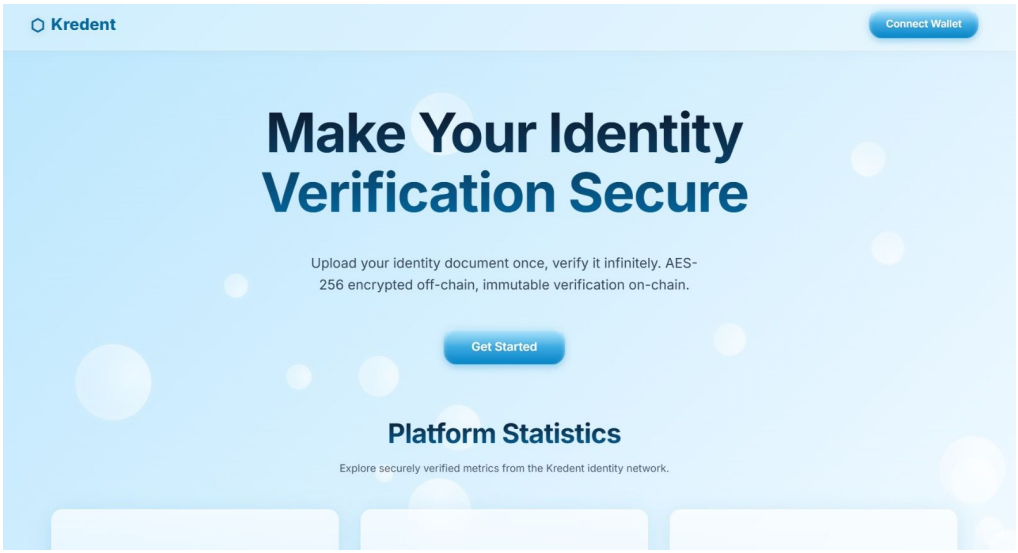

Figure 1: Frontend Dashboard - User Interface

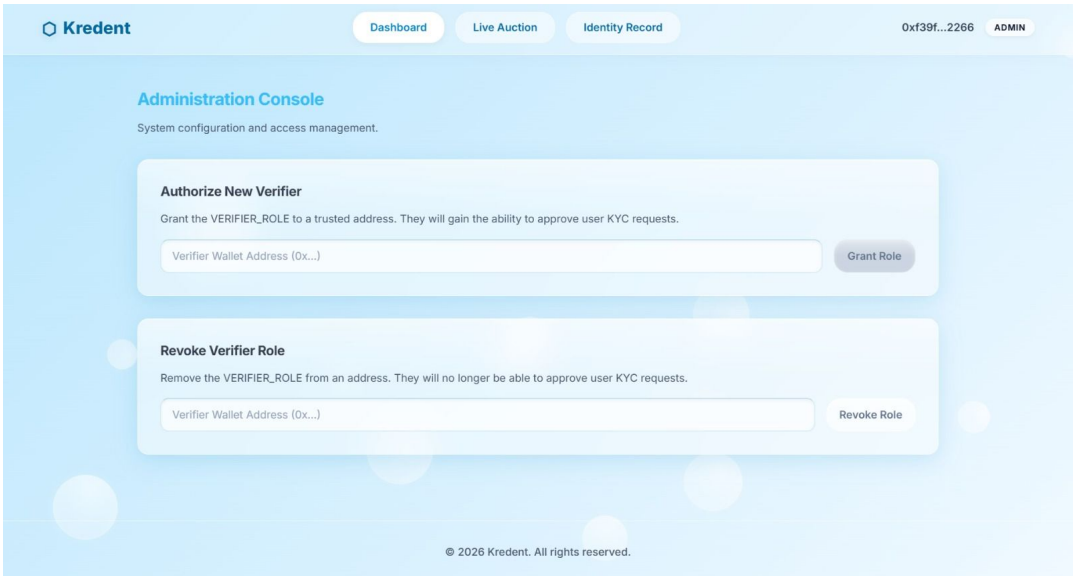

Figure 2: Frontend Dashboard - Verification Interface

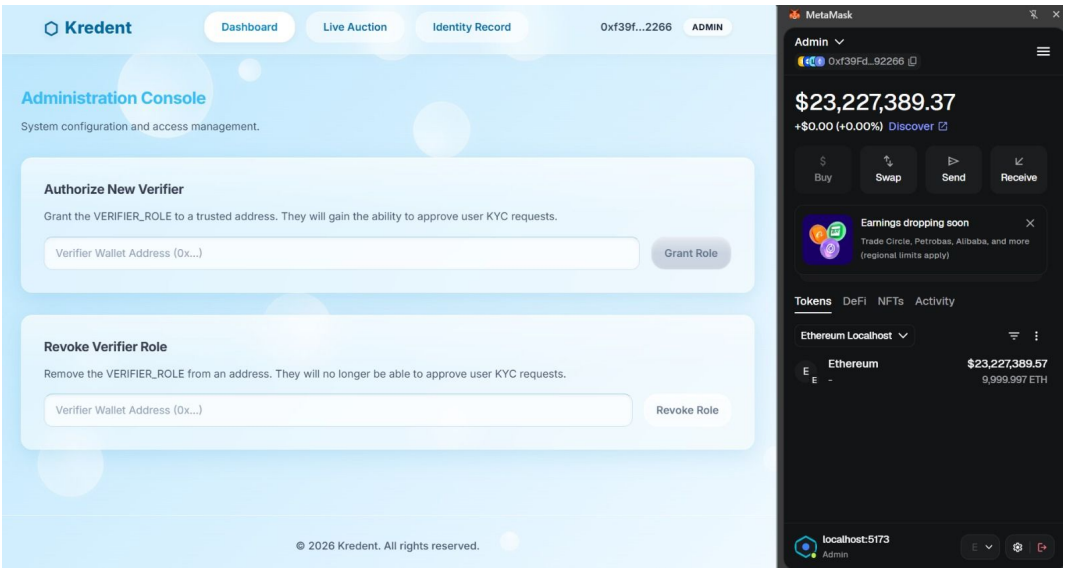

Figure 3: Frontend Dashboard - Auction Interface

2.2 Backend

The backend uses Express.js and MongoDB.

Responsibilities:

- AES encryption/decryption
- MongoDB document storage
- API routing
- Identity document retrieval

2.3 Blockchain Layer

Smart contracts are developed using Solidity and Hardhat.

Contracts:

- IdentityVerifier.sol
- KYCGatedAuction.sol

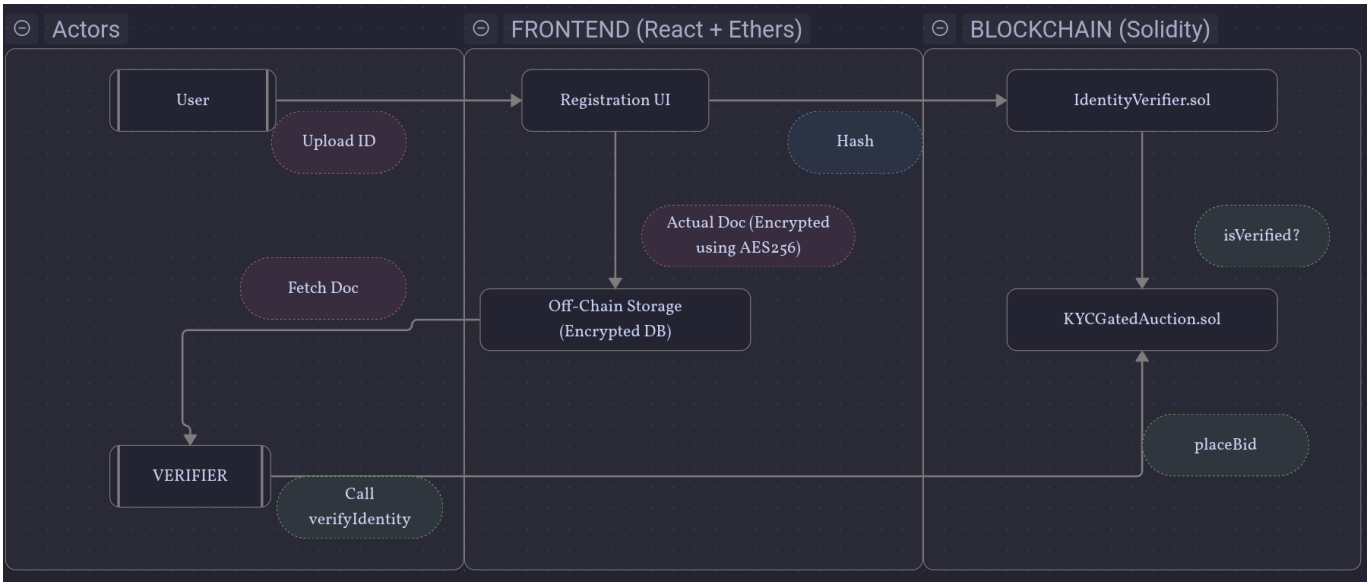

Figure 4: Overall System Architecture

3. Smart Contract Design

3.1 IdentityVerifier Contract

The IdentityVerifier contract manages registration, verification, and revocation of identities.

3.2 Functions

Function	Purpose
registerIdentity(bytes32)	Stores keccak256 document hash on-chain
verifyIdentity(address)	Marks identity as verified
revokeIdentity(address)	Revokes verified identity
addVerifier(address)	Adds authorised verifier
removeVerifier(address)	Removes verifier
isVerified(address)	Returns verification status

3.3 Access Control

The project uses OpenZeppelin AccessControl.

Roles:

- DEFAULT_ADMIN_ROLE
- VERIFIER_ROLE

3.4 KYC Gated Auction

Only verified users can place bids.

```
modifier onlyVerified() {
    require(verifier.isVerified(msg.sender),
            "KYC required");
    _;
}
```

4. Security Features

4.1 Reentrancy Protection

The KYCGatedAuction contract uses OpenZeppelin ReentrancyGuard.

Protected functions:

- placeBid()
- withdrawRefund()
- withdrawProceeds()

4.2 Input Validation

- Zero address validation
- Empty hash prevention
- Role-based restrictions
- Auction state validation

4.3 Encryption

AES-256-CBC encryption secures user documents before MongoDB storage.

5. GDPR and Privacy Compliance

This project follows a hash-only storage architecture.

5.1 Stored On-Chain

- keccak256 document hash
- Verification status

- Verifier address
- Verification timestamp

5.2 Stored Off-Chain

- Aadhaar card
- Passport
- Driving licence
- Personal details
- Biometric information

6. Database Design

MongoDB stores encrypted user documents.

7. Frontend Implementation

Technologies used:

- React
- Vite
- Ethers.js
- MetaMask
- CSS

7.1 Wallet Integration

MetaMask enables blockchain interaction and transaction signing.

8. Testing and Validation

The project includes comprehensive Hardhat tests.

8.1 Test Cases

- Non-verifier cannot approve identities
- Unregistered user is not verified
- Revoking identity updates status

- Unverified auction bid reverts
- Verified users can bid successfully

8.2 Coverage Report

```
IdentityVerifier.sol : 100% line coverage
KYCGatedAuction.sol : 100% line coverage
Overall Coverage     : 100%
```

9. Gas Optimisation

9.1 Struct Packing Optimisation

The Identity struct was optimised by changing timestamp datatype from uint256 to uint64.

```
struct Identity {
    bytes32 document_hash;
    Status status;
    address verified_by;
    uint64 timestamp;
}
```

9.2 Gas Report

Function	Before	After
registerIdentity	70579	68447
verifyIdentity	53308	31240
revokeIdentity	31120	31137

Table 2: Gas Optimisation Results

The optimisation reduced gas consumption of verifyIdentity() by approximately 41.4%.

10. Results

The project successfully demonstrates:

- Secure decentralised identity verification
- GDPR compliant architecture
- Blockchain composability

- Secure access control
- MetaMask integration
- Smart contract testing and optimisation

11. Advantages

- Decentralised trust model
- Strong privacy protection
- Immutable verification records
- Reduced fraud possibilities
- Easy verifier management
- Reusable KYC verification

12. Limitations

- Requires MetaMask wallet
- Blockchain transaction fees
- Dependence on verifier integrity
- Public visibility of wallet addresses

13. Future Scope

Future improvements may include:

- Zero Knowledge Proof integration
- Multi-chain support
- Decentralised storage using IPFS
- AI based fraud detection
- Mobile application support
- NFT based identity credentials

14. Conclusion

The project successfully implements a decentralised identity verification platform combining blockchain transparency with user privacy. By storing only cryptographic hashes

on-chain and maintaining encrypted off-chain documents, the system achieves GDPR compliance while preserving trustlessness and immutability.

The integration of KYC gated auctions demonstrates blockchain composability and real-world applicability.

15. References

1. Ethereum Documentation
2. Solidity Documentation
3. Hardhat Documentation
4. OpenZeppelin Contracts Library
5. MongoDB Documentation
6. React Documentation
7. GDPR Article 17 Guidelines
8. Ethers.js Documentation